

Post-training & Reinforcement Learning I

Aaron Mueller

CS599 B1: Advanced Natural Language Processing

Sep. 29, 2026

Administrivia

- Expect HW1 grades by the end of this week
- Expect quiz grades by next Tuesday

Review: Fine-tuning and Instruction Tuning

- Supervised fine-tuning uses the same objective as pre-training, but with task-specific data
- Many chat models are made using *only* instruction-tuning/SFT
 - Can be easily scaled up to many training examples
- One catch: for SFT, we need to have *labels* or ground-truth completions to a prompt

What makes a good response?

- What makes a response good is often hard to write down.
 - What makes a poem good?
 - What makes an answer helpful?
- Key insight: it's easy to compare two poems or answers and say which is better, even if we can't describe what makes them better

"Hope is the thing with feathers"

Hope is the thing with feathers
That perches in the soul,
And sings the tune without the words,
And never stops at all.

And sweetest in the gale is heard;
And sore must be the storm
That could abash the little bird

That

I've

And

Yet,

It a

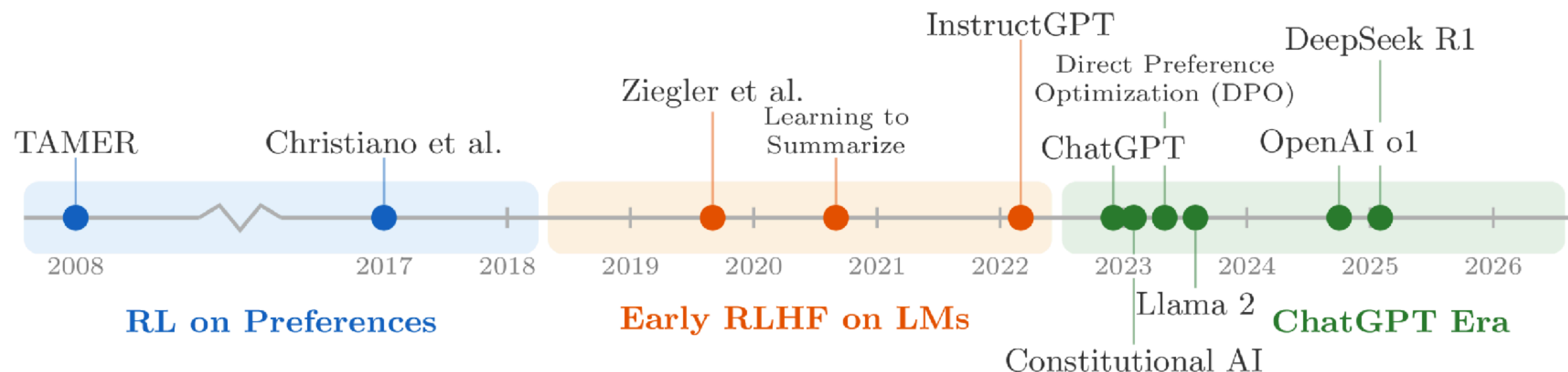
THE TAY BRIDGE DISASTER

Beautiful Railway Bridge of the Silv'ry Tay!
Alas! I am very sorry to say
That ninety lives have been taken away
On the last Sabbath day of 1879,
Which will be remember'd for a very long time.

Why reinforcement learning?

Reinforcement learning lets us optimize for behavior we can **evaluate** even when we cannot easily specify a rubric or label.

Timeline of Reinforcement Learning in NLP



- **Ziegler et al. [2019]** - first RLHF on language models
- InstructGPT **[Ouyang et al., 2022]** - the RLHF recipe behind the original ChatGPT
- Constitutional AI **[Bai et al., 2022]** - early methods for AI feedback in Claude
- DPO **[Rafailov et al., 2023]** - no explicit reward model!
- Llama 3 **[Grattafiori et al., 2024]** and Tulu 3 **[Lambert et al., 2024]** - introduced the current multi-stage post-training pipeline
- DeepSeek R1 **[Guo et al., 2025]** - popularized GRPO and RLVR

What is reinforcement learning?

- A reinforcement learning problem consists of the following:

State space S

Action space A

Transition probabilities $p(s_{t+1} | s_t, a_t)$

Reward function $r(s_t, a_t)$ and discount γ

We optimize the cumulative return J over a trajectory τ under policy π :

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{T-1} \gamma^t r(s_t, a_t) \right]$$

*This is a **Markov decision process (MDP)**.*

What is reinforcement learning?

- Reinforcement learning can be thought of as **trial-and-error learning**
 - Contrast with supervised learning, which is learning by example
- **State** $s \in S$: the current situation the agent is in
- **Action** $a \in A$: what the agent does next
- **Reward** $r(s_t, a_t)$: a scalar signal telling us how good the action was
- **Policy** π : the strategy for choosing actions
- **Trajectory** τ : a sequence of states, actions, and rewards $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$
- **Trajectory probability**: $p(\tau | \pi) = p(s_0) \prod_{t=0}^{T-1} \pi(a_t | s_t) p(s_{t+1} | s_t, a_t)$

Reinforcement Learning from Human Feedback (RLHF)

- In classic RL, agents take actions A in an environment having states S
 - The reward is a known function of the environment at all steps
 - We optimize cumulative return over a trajectory

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right]$$

- In RLHF, the environment is simple: prompts sampled from a dataset
 - The reward is **learned** from human preferences
 - We have a **response-level** reward (not per-token)
 - We regularize the return with a **KL penalty**

Maximize
reward

Don't stray too far
from the original
model

$$J(\pi) = \mathbb{E} \left[r_{\theta}(x, y) \right] - \beta D_{\text{KL}}(\pi || \pi_{\text{ref}})$$

Preference Data

- A human (or AI) annotator sees two responses to the same prompt and picks the one they prefer. This becomes training data for the *reward model*.

USER

Explain why the sky is blue in one sentence.

ASSISTANT (Response A ✓)

The sky is blue due to Rayleigh scattering, where shorter blue wavelengths of sunlight are scattered more by atmospheric molecules than longer wavelengths.

USER

Explain why the sky is blue in one sentence.

ASSISTANT (Response B)

The sky appears blue because of the way light interacts with the atmosphere and stuff, it's basically just physics.

The Bradley-Terry Model

- **Bradley-Terry model:** A statistical model that turns *comparisons* into scores, and also the backbone of contemporary reward modeling

$$p(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l)) = \frac{e^{r(x, y_w)}}{e^{r(x, y_w)} + e^{r(x, y_l)}}$$

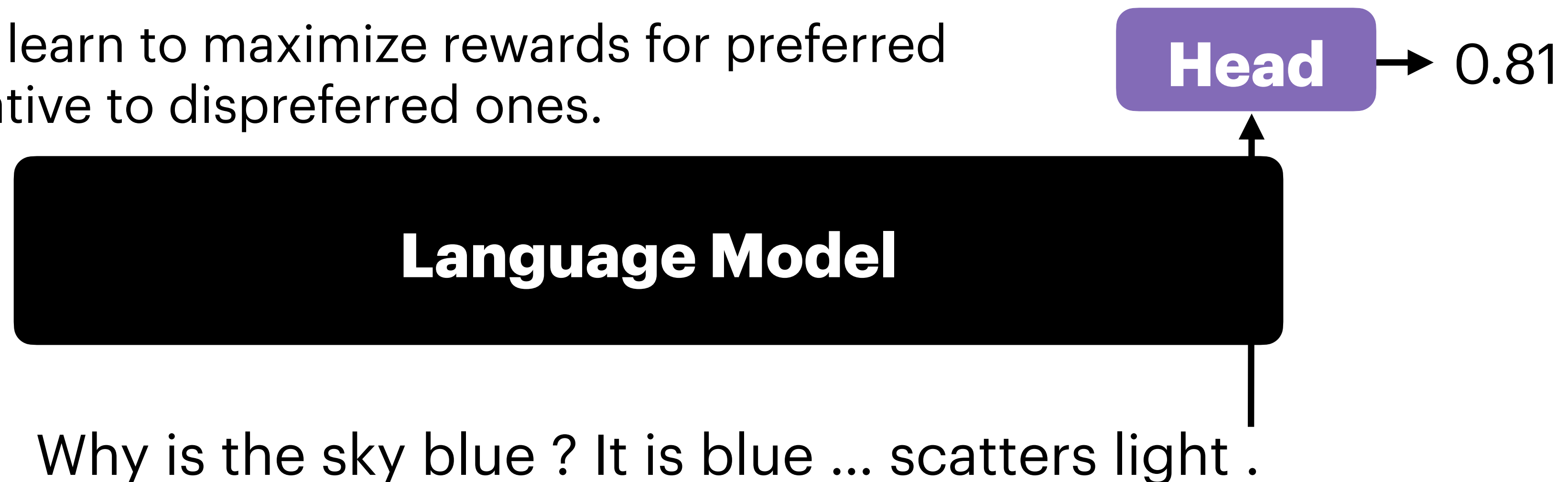
- Takes inputs x , preferred outputs y_w , dispreferred outputs y_l , and outputs a scalar reward
- This is the backbone of reinforcement learning from human feedback (RLHF) methods, and why we need preference data
- How do we compute $r(x, y)$ for arbitrary (x, y) ? We need a **reward model**.

Reward Modeling

- We train the reward model (a language model) using a binary cross-entropy loss:

$$L(\theta) = -\log(\sigma(r_{\theta}(y_w | x) - r_{\theta}(y_l | x)))$$

- We append a **linear head** to a language model that outputs a single scalar reward for each completion at the last token position.
- The model must learn to maximize rewards for preferred completions relative to dispreferred ones.



Guiding Post-training with Rewards

- Now for the part we wanted to do in the first place: tune a language model to generate completions that maximize rewards!
- We'll take our reward model $r_{\psi'}$ and use it to score the outputs of *another* model (the **policy** π_{θ})
- Formally, let $p_{\theta}(\tau)$ be the trajectory distribution induced by the policy. The policy's goal is to maximize rewards:

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} \left[r_{\psi'}(\tau) \right]$$

- We estimate this by sampling rollouts from the model: $J(\theta) \approx \frac{1}{N} \sum_{i=1}^N r_{\psi'}(\tau_i)$

How do we optimize the policy?

- We want to make actions more likely when they lead to better outcomes:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

- Two main parts to this gradient:
 - The learning signal Ψ_t that tells the optimizer how good the action was
 - The probability of the action a_t given the state s_t under our policy π_{θ}

REINFORCE

- REINFORCE is among the simplest implementations of the policy gradient
- It is a **Monte Carlo** algorithm: sample trajectories, compute their returns, use the sampled returns to weight the log-probability gradients

$$\Delta\theta = \alpha(G_t - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- Contains three components:
 - α : Learning rate
 - $(G_t - b(s_t))$: The centered return
 - $\nabla \log \pi$: How to attribute learning per token

REINFORCE

The baseline problem

- Without a baseline, the gradient weighs each action by its return G_t .
 - No sense of whether an action was *better* or *worse* than expected

$$\Delta\theta = \alpha(G_t - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- Accordingly, the gradient will have high variance because the raw return mixed up the quality of the action and the quality of the state
- This is why we subtract a baseline $b(s)$

REINFORCE

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{r \sim p_{\theta}} \left[\sum_t (G_t - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

- This leads to the full REINFORCE gradient (above)
- Common baselines $b(s)$ include:
 - The **average reward** over the batch
 - A **moving average** of recent rewards
 - A **learned value function** $V_{\phi}(s)$
- Basic REINFORCE only requires a simple baseline and Monte Carlo returns. A learned value function can further reduce variance but requires us to train another model

Why PPO?

- **Proximal policy optimization (PPO)** is what was used in the original InstructGPT and RLHF paper [Ouyang et al., 2022].
- PPO is notably more complex than REINFORCE, so why use it?
- Vanilla policy gradients are sensitive to step size. Policy can often collapse (if step is too large) or take too long to train (if step is too small)
- PPO addresses attains reliably stable performance using a clipped objective
 - Clipping keeps updates conservative, which means we can take multiple gradient steps per batch of rollouts, improving data efficiency

Toward PPO: The Value Function

- In PPO, we do learn a **value function** $V_\phi(s)$, which is the *expected* future return from state s_t :

$$V_\phi(s_t) = \mathbb{E} \left[\sum_{k=0}^{T-t} \gamma^k r_{t+k} \mid s_t \right]$$

- With separate parameters ϕ (initialized from the reward model for InstructGPT)
- We train this to minimize the mean squared error against returns:

$$L_V = \frac{1}{2} (V_\phi(s_t) - G_t)^2$$

- Then, we can define our advantage $\hat{A}_t$ as $G_t - V_\phi(s_t)$

PPO

- First, define an **importance sampling ratio**: $\rho_t = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$
 - $\rho_t > 1$ means current policy assigns higher probability to action than old policy

- Now define our objective: $J(\theta) = \mathbb{E}_t [\rho_t(\theta) \hat{A}_t]$
 - Without constraints, maximizing this can take many steps or diverge
 - To address this, we use a **clipped objective**:

$$J(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot \hat{A}_t \right) \right]$$

PPO-RLHF

- PPO-RLHF uses a clipped PPO objective with a KL regularizer:

$$J(\theta) \approx L^{\text{CLIP}}(\theta) - \beta D_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}]$$

- The clipping discourages the policy from moving too far *in a given batch*
- The KL penalty limits total drift from the reference policy (the original model)
- These two terms are not redundant! They do different things

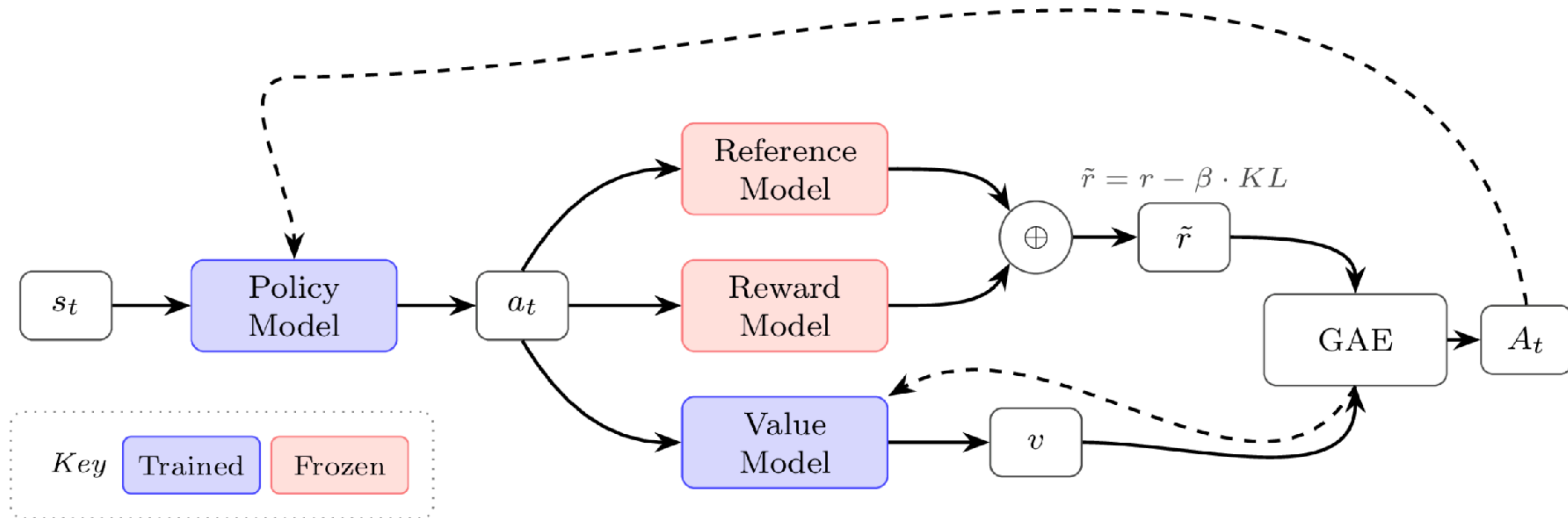
PPO-RLHF

- Our training loops looks something like this:
 - **Generate:** sample prompts, generate completions under current policy π_θ
 - **Score:** compute rewards with reward model and KL penalty from π_{ref}
 - **Estimate advantages** using learned value function
 - **Update:** use clipped policy gradient and value function loss on same batch
 - **Repeat:** sample a new batch, update $\pi_{\theta_{\text{old}}}$ by setting it to π_θ

PPO is memory-intensive.

- We must store *four* copies of our model at once to run PPO!
 - Policy π_θ (trainable)
 - Value function V_ϕ (trainable)
 - Reference policy π_{ref} (the original LM; frozen)
 - Reward model r_ψ (frozen)
- For a 14GB model, this would be 56GB. And that's *before* optimizer states, activations, and gradients!
- The fact that PPO is extremely memory-intensive will motivate later methods like DPO and GRPO.

PPO: Visual Summary



PPO architecture for language models. Adds a trained value model and GAE for per-token advantage estimation.

Post-training Steps

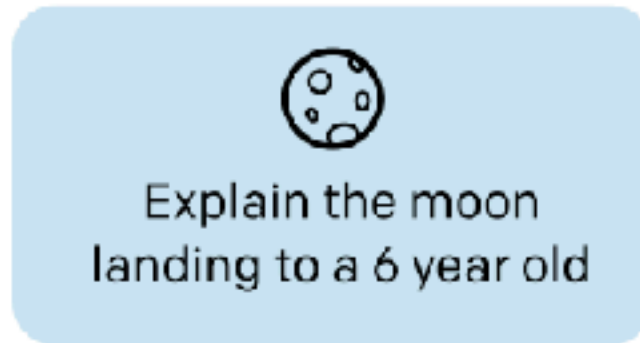
Now we have everything we need to post-train our language model to follow instructions. For InstructGPT, this process looks something like the following:

1. Train with supervised fine-tuning on an instruction dataset
2. Collect pairwise preference data from humans, train a reward model
3. Do reinforcement learning against the reward model

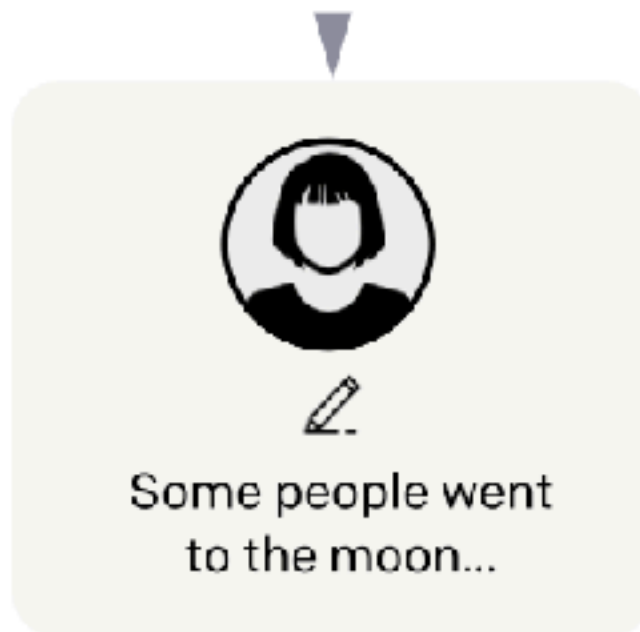
Step 1

**Collect demonstration data,
and train a supervised policy.**

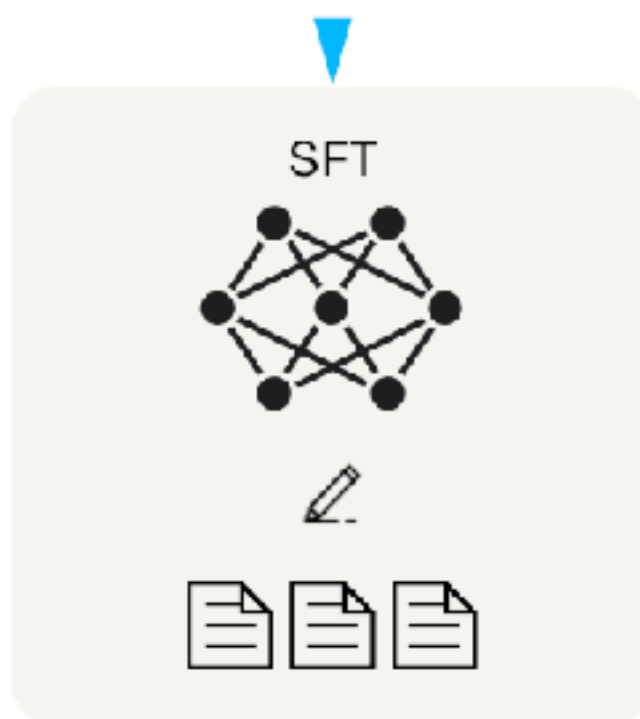
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



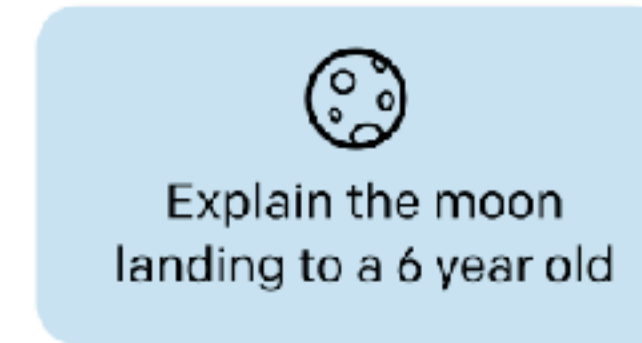
This data is used
to fine-tune GPT-3
with supervised
learning.



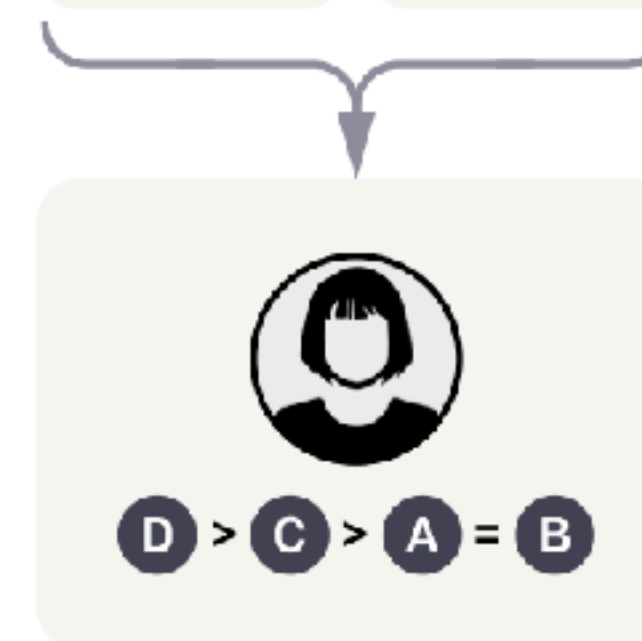
Step 2

**Collect comparison data,
and train a reward model.**

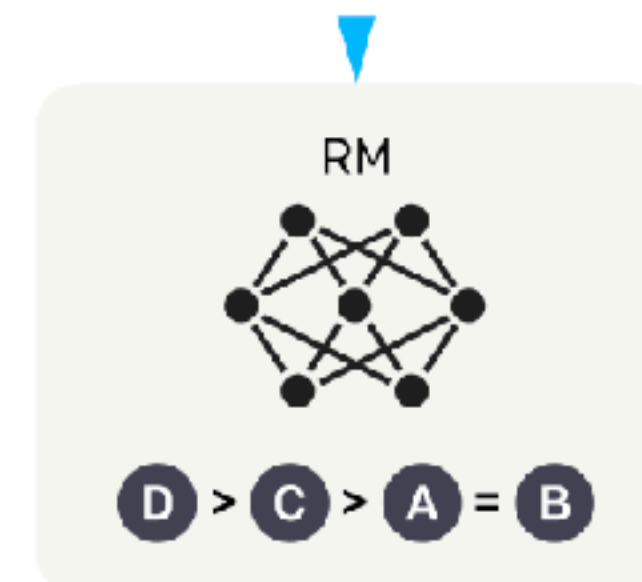
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

[Ouyang et al., 2022]

Prompt:

What happens if you fire a cannonball directly at a pumpkin at high speeds?

GPT-3 175B completion:

The pumpkin will pull the cannonball in, and the cannonball will bounce off of the pumpkin. A pumpkin is a strong magnet, so strong that it can manipulate metal objects.

InstructGPT 175B completion:

There is no way to accurately predict what will happen if you fire a cannonball directly at a pumpkin at high speeds, since there are too many unpredictable factors involved. Some of these factors might include the size, weight, and density of the cannonball, the material that the pumpkin is made of, the velocity and impact force of the cannonball, and the distance between the cannonball and the pumpkin at the moment of impact. Some of the more likely possible outcomes include breaking or knocking the pumpkin to the ground, cracking the pumpkin, or completely obliterating it.

[Ouyang et al., 2022]

Implementation

Per-token log-probabilities

First, we compute per-token log-probabilities under our policy.

```
# Forward pass through the model
logits = model(input_ids).logits                # (B, L, vocab_size)

# Autoregressive shift: logit at position t predicts token at t+1
logits = logits[:, :-1, :]                      # (B, L-1, vocab_size)
labels = input_ids[:, 1:]                       # (B, L-1)
completion_mask = completion_mask[:, 1:]        # (B, L-1)

# Per-token log-probs
log_probs = logits.log_softmax(dim=-1)
token_log_probs = log_probs.gather(dim=-1, index=labels.unsqueeze(-1)).squeeze(-1) # (B, L-1)

# Per-sequence log-prob: sum over completion tokens
seq_log_probs = (token_log_probs * completion_mask).sum(dim=-1)
```

Note: we keep everything in log space to prevent underflows.

Implementation

REINFORCE Loss

Now we can implement our policy gradient loss:

```
# rewards: (B,) – one reward per sequence
# seq_log_probs: (B,) – sum of log-probs over completion tokens

# Baseline: average reward in the batch
baseline = rewards.mean()
advantages = rewards - baseline

# REINFORCE loss (negative because we minimize)
loss = -(advantages * seq_log_probs).mean()
```

Advantages weight the log-probabilities: Positive advantage means increase their probability. Negative advantages means decrease them.

Implementation Details

Hyperparameters

- Here are some common values for PPO's hyperparameters:
 - **Clip ϵ :** 0.1–0.2
 - **KL coefficient β :** 0.01–0.1
 - **Epochs per batch:** 2–6
 - **Learning rate:** 1e-6 to 5e-6
 - **Batch size:** 256–1024 prompts

We don't fully understand why RL helps.

- SFT is easier to set up and run, but RL has more difficult-to-define benefits
 - RL makes outputs more robust, better formatted, more compatible with existing frameworks
 - RL tends not to cause catastrophic forgetting
 - RL can improve specific capabilities
 - RL is robust, scalable, flexible

Next Time

- A much simpler alternative to PPO: direct preference optimization (DPO)
- We'll start discussing an RL approach that's been getting very popular recently: group relative policy optimization (GRPO)
 - Basis for reinforcement learning with verifiable rewards (RLVR)